



# Redovisning Cheat Sheet — SchoolAppVS + Hogstadiet

## Labb 3 — EF Core Console App

---



### PROJEKTSTRUKTUR

```
SchoolAppVS (Visual Studio projekt)
|
|— Models/           ← C#-klasser som speglar SQL-tabeller
|   |— Title.cs
|   |— Staff.cs
|   |— Class.cs
|   |— Student.cs
|   |— Subject.cs
|   |— Grade.cs
|
|— EFQueries.cs      ← Alla databasmetoder samlade här
|— SchoolContext.cs ← Bryggan mellan C# och SQL Server
|— Program.cs        ← Meny och navigation

SQL Server databas: Hogstadiet
```



### NYCKELBEGREPP — GENERELLA

| Begrepp                          | Förklaring                                                                                       |
|----------------------------------|--------------------------------------------------------------------------------------------------|
| EF Core                          | Object-Relational Mapper — översätter mellan C#-objekt och databastabeller automatiskt           |
| ORM                              | Object-Relational Mapper — det EF Core är. Man skriver C# istället för SQL                       |
| Namespace                        | En behållare för klasser. Måste matcha projektnamnet i VS                                        |
| <code>using</code> (överst)      | Importerar verktygspaket till hela filen. T.ex. <code>using Microsoft.EntityFrameworkCore</code> |
| <code>using var</code> (i metod) | Skapar en resurs och stänger den automatiskt när metoden är klar — t.ex. databasanslutning       |

| Begrepp                   | Förklaring                                                                                               |
|---------------------------|----------------------------------------------------------------------------------------------------------|
| <code>public</code>       | Klassen/metoden är synlig överallt — krävs av EF Core                                                    |
| <code>var</code>          | Låter kompilatorn räkna ut typen automatiskt. <code>var x = 5</code> är samma som <code>int x = 5</code> |
| <code>string.Empty</code> | Tomt strängvärde <code>""</code> — används för att undvika null-varningar                                |

## SCHOOLCONTEXT.CS

```
csharp
public class SchoolContext : DbContext
```

## Nyckelord

| Begrepp                                  | Förklaring                                                                               |
|------------------------------------------|------------------------------------------------------------------------------------------|
| <code>DbContext</code>                   | EF Cores brygga till databasen. SchoolContext ärver från den                             |
| <code>DbSet&lt;T&gt;</code>              | Representerar en tabell. <code>DbSet&lt;Student&gt;</code> = Students-tabellen           |
| <code>OnConfiguring</code>               | Metoden där man talar om VAR databasen finns                                             |
| <code>UseSqlServer(...)</code>           | Talar om VILKEN typ av databas (SQL Server specifikt)                                    |
| Connection String                        | Adressen till databasen. <code>Database=Hogstadiet</code> måste matcha SQL Server-namnet |
| <code>TrustServerCertificate=True</code> | Behövs lokalt för att undvika SSL-fel                                                    |

## Viktigt att säga

*"SchoolContext ärver från DbContext som är EF Cores brygga till databasen. Varje DbSet representerar en tabell. I OnConfiguring talar vi om var databasen finns via en connection string — databasnamnet måste matcha det faktiska namnet i SQL Server, i mitt fall Hogstadiet."*

## Varför en modell per tabell?

"EF Core behöver en C#-klass som speglar varje tabell. Varje property motsvarar en kolumn. Det kallas databas-mappning."

## Regler

- En tabell i SQL = En modellklass i C#
- Klassnamn är alltid **singular** (Student, inte Students)
- **public** på alla klasser — EF Core måste kunna läsa dem
- Properties måste matcha kolumnnamnen i SQL

## Annotation — undantag från regeln

```
csharp
[Column("Grade")]           // SQL-kolumnen heter "Grade"
public string GradeValue    // C#-propertyn heter "GradeValue"
{ get; set; }
```

"Här använde jag en Column-annotation eftersom property-namnet i C# och kolumnnamnet i SQL skiljer sig åt. `[Column("Grade")]` talar om för EF Core vilket kolumnnamn den ska leta efter."

## Nullable — **int?**

```
csharp
public int? TeacherId { get; set; } // kan vara null
public int? ClassId { get; set; }   // kan vara null
```

"Frågetecknet betyder att värdet får vara null — används när SQL-kolumnen inte har NOT NULL."

## EFQUERIES.CS — METODERNA

### Gemensamma begrepp för alla metoder

| Begrepp         | Förklaring                                                                                  |
|-----------------|---------------------------------------------------------------------------------------------|
| <b>Select()</b> | Projektion — väljer bara de kolumner man behöver. Minimerar data från databasen till minnet |

| Begrepp                             | Förklaring                                                                                           |
|-------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>Where()</code>                | Filter — motsvarar SQL:s WHERE-sats                                                                  |
| <code>ToList()</code>               | Hämtar all data till RAM-minnet. Används sparsamt — Select är bättre                                 |
| <code>AsQueryable()</code>          | Bygger frågan steg för steg innan den skickas till SQL                                               |
| <code>OrderBy()</code>              | Sorterar stigande — motsvarar ORDER BY ASC i SQL                                                     |
| <code>OrderByDescending()</code>    | Sorterar fallande — motsvarar ORDER BY DESC i SQL                                                    |
| <code>FirstOrDefault()</code>       | Hämtar första matchande rad, eller null om ingen finns. Säkrare än <code>First()</code> som kraschar |
| <code>Any()</code>                  | Kollar om en lista har några element — returnerar true/false                                         |
| Lambda <code>=&gt;</code>           | <code>s =&gt; s.FirstName</code> betyder "för varje s, ta s.FirstName"                               |
| Anonymt objekt <code>new { }</code> | Tillfälligt objekt utan egen klass — används när man bara ska visa data                              |

## 1. ObtainAllStudents()

**Syfte:** Hämtar alla elever med valbar sortering

```
csharp
var students = context.Students
    .Select(s => new { s.FirstName, s.LastName });
```

## Varför if-satser istället för switch?

"Jag valde if-satser här eftersom jag jämför två variabler samtidigt — `sortVal` och `sortOrder`. Switch case passar bättre när man jämför en variabel mot flera fasta värden, som i huvudmenyn."

## 2. FetchStudentsInSpecificClass()

**Syfte:** Visar klasser → användaren väljer → visar elever i klassen

```
csharp
```

```
var students = context.Students
    .Where(s => s.ClassId == classId)
    .Select(s => new { s.FirstName, s.LastName });
```

## Where + Select:

*"Where filtrerar i databasen, Select väljer kolumner. EF Core skickar allt som en enda SQL-fråga — inget filtreras i C#-minnet."*

## 3. AddStaff()

**Syfte:** Lägger till ny personal i databasen

```
csharp

var newStaff = new Staff { FirstName = firstName, ... };
context.Staff.Add(newStaff);
context.SaveChanges();
```

### Begrepp

### Förklaring

|                                    |                                                          |
|------------------------------------|----------------------------------------------------------|
| <code>new Staff { }</code>         | Skapar ett C#-objekt som speglar en rad i Staff-tabellen |
| <code>context.Staff.Add()</code>   | Registrerar objektet hos EF Core — sparar inte än        |
| <code>context.SaveChanges()</code> | FÖRST HÄR körs INSERT INTO mot databasen                 |

*"`SaveChanges()` är kritisk — EF Core samlar ihop alla ändringar och skickar dem till databasen först när den anropas. Som att fylla i en beställningsblankett och sedan trycka skicka."*

## 4. ObtainAllStaff()

**Syfte:** Visar all personal eller filtrerat per titel

**Två queries — if/else:**

*"Uppgiften kräver antingen alla anställda eller filtrerat per kategori. Jag löste det med en if/else — om användaren väljer 1 visas all personal, annars väljer de en titel och får filtrerat resultat."*

## 5. GradesLastMonth()

**Syfte:** Visar betyg satta senaste månaden med elevnamn och ämne

```
csharp

var oneMonthAgo = DateTime.Now.AddMonths(-1);

var grades = context.Grades
    .Where(g => g.GradeDate >= oneMonthAgo)
    .Select(g => new {
        StudentName = context.Students
            .Where(s => s.Id == g.StudentId)
            .Select(s => s.FirstName + " " + s.LastName)
            .FirstOrDefault(),
        SubjectName = context.Subjects
            .Where(s => s.Id == g.SubjectId)
            .Select(s => s.Name)
            .FirstOrDefault(),
        g.GradeValue,
        g.GradeDate
    });
```

### Begrepp

### Förklaring

**DateTime.Now.AddMonths(-1)**

Räknar ut datumet för en månad sedan

**FirstOrDefault()**

Hämtar första matchande — null om ingen finns

*"Utan navigation properties hämtar jag elevnamn och ämnesnamn direkt inne i Select via separata delfrågor mot Students- och Subjects-tabellerna."*

## 6. AverageGradePerSubject()

**Syfte:** Snittbetyg, högsta och lägsta betyg per ämne

```
csharp
```

```
// Bokstav → Siffra för uträkning
var gradeNumbers = sub.Grades.Select(g => g switch
{
    "A" => 5, "B" => 4, "C" => 3, "D" => 2, "E" => 1, _ => 0
});

double avg = gradeNumbers.Average();
```

| Begrepp                 | Förklaring                                                     |
|-------------------------|----------------------------------------------------------------|
| Switch expression       | Omvandlar bokstavs-betyg till siffror för matematisk uträkning |
| <code>.Average()</code> | Räknar medelvärde                                              |
| <code>{avg:F1}</code>   | Formaterar till en decimal — t.ex. 3.7                         |
| <code>_ =&gt; 0</code>  | Default-värde om betyget inte matchar något                    |

## Varför omvandla?

*"Betyg lagras som bokstäver i databasen. För att räkna ett matematiskt medelvärde omvandlar jag dem till siffror med en switch expression."*

## 7. AddNewStudent() — Stored Procedure

**Syfte:** Läger till ny elev via en stored procedure i SQL Server

```
csharp

context.Database.ExecuteSqlRaw(
    "EXEC AddNewStudent @p0, @p1, @p2, @p3, @p4, @p5",
    firstName, lastName, dateOfBirth, gender, socialSecurityNo, classId);
```

| Begrepp                      | Förklaring                                                              |
|------------------------------|-------------------------------------------------------------------------|
| Stored Procedure             | Förbyggd SQL-rutin lagrad direkt i databasen. Anropas som en funktion   |
| <code>ExecuteSqlRaw()</code> | EF Cores sätt att anropa stored procedures direkt                       |
| <code>@p0, @p1...</code>     | Parametrar som skickas säkert — skyddar mot SQL injection               |
| SQL Injection                | Attack där elakartad input kan manipulera eller radera data i databasen |

"AddNewStudent löses med en stored procedure enligt uppgiftskravet. Proceduren skapas i SQL Server och anropas från C# via ExecuteSqlRaw. Parametrarna skickas separat vilket skyddar mot SQL injection."

## PROGRAM.CS — NAVIGATION

```
csharp

bool running = true;
while (running)          // håller menyn igång
{
    switch (choice)       // jämför ett val mot flera fasta värden
    {
        case "1": queries.ObtainAllStudents(); break;
        ...
        case "0": running = false; break;
        default: Console.WriteLine("Ogiltigt val"); break;
    }
}
```

| Begrepp                        | Förklaring                                                                           |
|--------------------------------|--------------------------------------------------------------------------------------|
| <code>while (running)</code>   | Loop som håller menyn igång tills användaren väljer 0                                |
| Switch case                    | Passar när man jämför EN variabel mot flera fasta värden                             |
| <code>Console.ReadKey()</code> | Pausar programmet tills användaren trycker en tangent — annars försvinner resultatet |
| <code>Console.Clear()</code>   | Rensar konsolen — används i while-loopen för ren meny varje varv                     |

## FELHANTERING

```
csharp
```



```
// int.Parse - kraschar om användaren skriver text
int.Parse(Console.ReadLine())

// int.TryParse - säker variant
if (!int.TryParse(Console.ReadLine(), out int titleId))
{
    Console.WriteLine("Ogiltigt val!");
    return; // avbryter metoden, går tillbaka till menyn
}
```

## Begrepp

## Förklaring

|                             |                                                            |
|-----------------------------|------------------------------------------------------------|
| <code>int.Parse()</code>    | Kraschar om input inte är en siffra                        |
| <code>int.TryParse()</code> | Returnerar false istället för att krascha                  |
| <code>return</code> i void  | Avbryter metoden och återgår till anroparen (menyn)        |
| <code>out int</code>        | Nyckelord som talar om att variabeln skapas inuti TryParse |

## SQL — VIKTIGA KOPPLINGAR

### SQL

### C# / EF Core

|                                    |                                                                       |
|------------------------------------|-----------------------------------------------------------------------|
| <code>CREATE TABLE Students</code> | <code>public class Student</code> + <code>DbSet&lt;Student&gt;</code> |
| <code>WHERE</code>                 | <code>.Where(s =&gt; s.Id == id)</code>                               |
| <code>ORDER BY ASC</code>          | <code>.OrderBy(s =&gt; s.Name)</code>                                 |
| <code>ORDER BY DESC</code>         | <code>.OrderByDescending(s =&gt; s.Name)</code>                       |
| <code>SELECT kolumn</code>         | <code>.Select(s =&gt; new { s.Name })</code>                          |
| <code>INSERT INTO</code>           | <code>context.Add()</code> + <code>context.SaveChanges()</code>       |
| <code>EXEC StoredProcedure</code>  | <code>context.Database.ExecuteSqlRaw("EXEC ...")</code>               |

## "Varför EF Core istället för vanlig SQL?"

*"EF Core låter mig skriva C# istället för SQL-strängar. Det är typsäkert, lättare att underhålla och skyddar automatiskt mot SQL injection."*

## "Vad är skillnaden mellan Select och ToList?"

*"Select är en projektion — den väljer bara de kolumner jag behöver och minimerar datamängden. ToList hämtar allt till RAM-minnet. Jag använder Select för att vara effektiv."*

## "Varför stored procedure för AddNewStudent?"

*"Det var ett krav i uppgiften. Stored procedures lagras i databasen och kan återanvändas. De ger också bättre säkerhet och prestanda."*

## "Vad är SQL injection?"

*"En attack där elakartad input kan manipulera SQL-frågor. Genom att använda parametrar (@p0, @p1) behandlas input alltid som data — aldrig som kod."*

## "Varför public på modellklasserna?"

*"EF Core måste kunna läsa klasserna när den mappar mot databasen. Internal begränsar synligheten för mycket."*

## "Vad är skillnaden mellan de två using?"

*"`using` högst upp importerar verktygspaket till hela filen. `using var` inuti en metod garanterar att databasanslutningen stängs automatiskt när metoden är klar — det kallas disposable pattern."*